高级语言程序设计

实验报告

南开大学 计算机大类
荆誉熙
2511181
工科试验班 3-2 班
2026 年 5 月 10 日

目录

一、作业题目 3
二、开发软件 3
三.、课题要求 3
四、主要流程 3
1. 整体流程 3
2. 算法或公式 4
3. 单元测试 5
五、单元测试 5
六、收获 5
1. 组合优于继承 5
2. 状态机设计 6
3. 单元测试与时序调试 6
七. AI 工具使用披露 6
八、附录 7

一、作业题目

《深海回响》

基于 C++与 SFML 的 2D 动作游戏。玩家扮演《明日方舟》里面的角色，与持双手剑的 Boss 进行单挑战斗，融合《只狼》架势条与弹反的战斗机制。

二、开发软件

Visual Studio + SFML 2.6.1

三.、课题要求

1. 面向对象。
2. 单元测试。
3. 状态机设计。
4. 图形化界面与帧动画。
5. 多模块协作（音频/特效/输入/物理）。

四、主要流程

1. 整体流程

实现思路:

游戏采用主循环 + 组合模式架构。核心类 Game 持有 Player 和 Boss，两者均通过组合方式持有功能组件（弹反、架势、动画、攻击），不使用继承耦合。

主循环每帧执行三步:

```
while {
handleInput();
update(dt);
render();
}
```

类组合关系图：

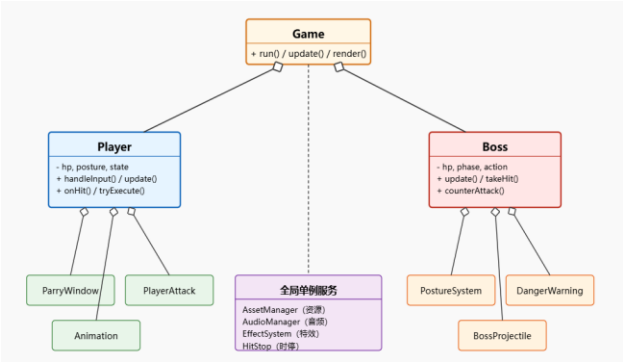

图 1 系统类图

游戏使用相同的"组合"模式构造不同角色。例如玩家和 Boss 都需要"动画"功能时，仅需各自持有一个 Animation 实例，无需通过继承复用代码。

2. 算法或公式

（1）完美弹反判定

ParryWindow 组件在玩家按 J 键时启动计时，根据时间窗口返回三种结果：

1) Parry
2) Block
3) None

按下 j 键后又 0.15 秒的窗口，参考《只狼》的弹反判定。

（2）架势条数学模型

架势值随时间变化的更新规则：

$$P_{t+1} = P_t + D_{posture} \quad \text{（受击时）}$$

$$P_{t+1} = P_t + D_{posture} \quad \text{（受击时）}$$

$$P_{t+1} = \max(0,\ P_t - r \cdot \mu(h) \cdot \Delta t) \quad \text{（不受击时）}$$

其中：

- $r = 15$（每秒基础回复速率）
- $\mu(h) = 0.3 + 0.7h$（HP 比例系数，$h \in [0,1]$）

血量越低，回复越慢——HP=0 时回复仅为基础值的 30%。

（3）Boss 多阶段 AI

Boss 根据当前 HP 切换三个阶段，各阶段移动速度与招式概率如下：

阶段	HP 区间	速度(px/s)	Slash	Dash	Danger
1	100% ~ 66%	80	40%	30%	10%
2	66% ~ 33%	120	40%	35%	20%
3	< 33%	180	30%	40%	30%

（4）破防加成机制

为增强架势条战略价值，原创设计 5 秒"破防恢复加成窗口"：

$D_{final} =$

$3D_{base}$，$\text{若}staggerBonus\text{激活}$

D_{base}，其他情况

Boss 架势条满 → 破防 2.5 秒 → 未处决则恢复 → 开启 5 秒加成窗口 → 期间任何攻击 3 倍伤害。

3. 单元测试

针对最关键的 ParryWindow 组件设计 5 个测试用例，覆盖弹反窗口判定、临界值、冷却机制等核心逻辑。

五、单元测试

测试案例定义:

输入	输出	目的
startBlock(); update(0.10)	PARRY	弹反窗口内受击
startBlock(); update(0.30)	BLOCK	超出弹反窗口
未按 J 键; update(0.10)	NONE	未格挡受击
startBlock(); update(0.15)	PARRY	边界临界值（=0.15s）

startBlock();
stopBlock();
startBlock()
	isBlocking()=false	冷却内重复格挡被拒绝

测试结果:

5 个测试用例全部通过。开发过程中曾出现一次未通过的情况：边界值 0.15s 被误判为 BLOCK，分析原因是判定条件写成了<而非<=。修改后重新测试通过。

六、收获

1. 组合优于继承

最初设计时考虑让 Player 继承"带格挡的角色基类"，让 Boss 继承"带架势条的角色基类"。但这会带来矛盾——如果想让 Boss 也能弹反玩家攻击怎么办？多重继承会带来菱形问题。

最终采用**组合模式**:

```
class Player {
    ParryWindow parry; // 组合一个弹反组件
    PostureSystem posture; // 组合一个架势条组件
```

```
    PlayerAttack attackSys; // 组合一个攻击组件
};
```

未来想让 Boss 也能弹反，只需添加一行 ParryWindow parry; 即可，无需修改任何继承关系。

2. 状态机设计

Boss AI 包含 8 种动作（IDLE / MOVE / SLASH / DASH / DANGER_SMASH / HURT / STUNNED / SWEEP），如果用大量 if-else 实现会难以维护。

采用**有限状态机**后，每个状态独立处理，新增招式只需添加一个 case 分支：

```
switch (action) {
    case BossAction::SLASH: /* 斩击逻辑 */ break;
    case BossAction::DASH: /* 冲刺逻辑 */ break;
    case BossAction::DANGER_SMASH: /* 不可格挡 */ break;
    case BossAction::SWEEP: /* 横扫霸体 */ break;
    default: break;
    }
```

整个开发过程中，从最初的 3 种招式扩展到 8 种，主循环代码几乎未变动。

3. 单元测试与时序调试

ParryWindow 在开发过程中曾出现 3 次 Bug：

- 临界值 0.15s 被误判为 BLOCK（条件写错）
- 松开 J 立刻再按意外触发 PARRY（缺少冷却）
- 静态变量跨菜单状态污染（应改为成员变量）

每次 Bug 都因有单元测试**几分钟内复现并修复**。如果只靠手动游戏测试很难复现这些时序问题。

```
ParryWindow p(0.15f, 0.4f);
p.startBlock();
p.update(0.15f);
assert(p.judge() == ParryWindow::Result::PARRY);
```

测试代码就是这一行，但它救了我大量的 debug 时间。

七. AI 工具使用披露

相关工具:deepseek-V3，Claude Opus 4.5，GitHub Copilot

使用位置:

1. 技能特效的生成，项目里面 EffectSystem.h 文件里面的代码，以及调用 SFML 绘制技能的特效
2. 生成对应的编程思路，规划代码运行的 V0.5，V1.0，V2.0，V3.0 的步骤，并优化了 Game.cpp,Boss.cpp,player.cpp 里面一些不必要的代码
3. 使用 GitHub Copilot 解决有错误的代码

八、附录

源代码:https://github.com/commoncreate/homework/tree/master

GitHub 提交次数 ≥ 3 次

B 站视频【【南开大学 26C++】新时代打铁?0 潜中杯游戏实验测评】https://www.bilibili.com/video/BV1im5j6LEyQ?vd_source=01d3b758483ef96e83a210d1df11abd3

(时长:7min46s)